```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow
import tensorflow.keras.datasets as datasets
import tensorflow.keras.layers as layers
import tensorflow.keras.models as models
from tensorflow.keras.utils import to_categorical, plot_model
from tensorflow.keras.callbacks import TensorBoard

# 1. Load CIFAR-100 (as in your Exp05)
(x_train, y_train), (x_test, y_test) = datasets.cifar100.load_data()

x_train = x_train.astype("float32") / 255.0
x_test  = x_test.astype("float32")  / 255.0

num_labels  = len(np.unique(y_train))   # 100
image_size  = x_train.shape[1]          # 32
input_shape = (image_size, image_size, 3)

y_train = to_categorical(y_train)
y_test  = to_categorical(y_test)

batch_size = 128
epochs     = 10
optim      = tensorflow.keras.optimizers.Adagrad(learning_rate=0.01)

# 2. AlexNet Architecture (exactly from your Exp05 Cell 9)
model = models.Sequential()
model.add(layers.Convolution2D(96, (11,11), input_shape=input_shape, activation="relu"))
model.add(layers.ZeroPadding2D((1,1)))
model.add(layers.MaxPooling2D(pool_size=(3,3), strides=2))

model.add(layers.Convolution2D(256, (5,5), activation="relu"))
model.add(layers.ZeroPadding2D(padding=(1,1)))
model.add(layers.MaxPooling2D(pool_size=(3,3), strides=1))

model.add(layers.Convolution2D(384, (3,3), activation="relu"))
model.add(layers.Convolution2D(384, (3,3), activation="relu"))
model.add(layers.Convolution2D(256, (3,3), activation="relu"))
model.add(layers.ZeroPadding2D(padding=(1,1)))
model.add(layers.MaxPooling2D(pool_size=(3,3), strides=1))

model.add(layers.Flatten())
model.add(layers.Dense(4096, activation="relu"))
model.add(layers.Dropout(0.6))
model.add(layers.Dense(4096, activation="relu"))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(num_labels, activation="softmax"))

model.summary()

# 3. Compile & Train
callbacks = [TensorBoard(log_dir="./logs")]
model.compile(loss="categorical_crossentropy",
              optimizer=optim,
              metrics=["accuracy"])

model.fit(x_train, y_train,
          batch_size=batch_size,
          epochs=epochs,
          validation_split=0.2,
          callbacks=callbacks)

# 4. Evaluate
_, acc = model.evaluate(x_test, y_test, batch_size=batch_size, verbose=1)
print(f"\nTest Accuracy: {100.0 * acc:.2f}%")
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz
169001437/169001437 ━━━━━━━━━━━━━━━━━━━━ 17s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 22, 22, 96)	34,944
zero_padding2d (ZeroPadding2D)	(None, 24, 24, 96)	0
max_pooling2d (MaxPooling2D)	(None, 11, 11, 96)	0
conv2d_1 (Conv2D)	(None, 7, 7, 256)	614,656
zero_padding2d_1 (ZeroPadding2D)	(None, 9, 9, 256)	0
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 256)	0
conv2d_2 (Conv2D)	(None, 5, 5, 384)	885,120
conv2d_3 (Conv2D)	(None, 3, 3, 384)	1,327,488
conv2d_4 (Conv2D)	(None, 1, 1, 256)	884,992
zero_padding2d_2 (ZeroPadding2D)	(None, 3, 3, 256)	0
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 256)	0
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 4096)	1,052,672
dropout (Dropout)	(None, 4096)	0
dense_1 (Dense)	(None, 4096)	16,781,312
dropout_1 (Dropout)	(None, 4096)	0
dense_2 (Dense)	(None, 100)	409,700

```
 Total params: 21,990,884 (83.89 MB)
 Trainable params: 21,990,884 (83.89 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 29s 58ms/step - accuracy: 0.0106 - loss: 4.6046 - val_accuracy: 0.0139 - val_loss: 4.6029
Epoch 2/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 9s 30ms/step - accuracy: 0.0163 - loss: 4.5776 - val_accuracy: 0.0199 - val_loss: 4.4588
Epoch 3/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.0247 - loss: 4.4082 - val_accuracy: 0.0226 - val_loss: 4.4264
Epoch 4/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 30ms/step - accuracy: 0.0313 - loss: 4.3159 - val_accuracy: 0.0445 - val_loss: 4.2324
Epoch 5/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 31ms/step - accuracy: 0.0416 - loss: 4.2158 - val_accuracy: 0.0580 - val_loss: 4.1269
Epoch 6/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.0540 - loss: 4.1324 - val_accuracy: 0.0591 - val_loss: 4.1995
```

Start coding or generate with AI.

```
Epoch 8/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.0862 - loss: 3.9641 - val_accuracy: 0.1014 - val_loss: 3.8687
Epoch 9/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.1011 - loss: 3.8691 - val_accuracy: 0.1049 - val_loss: 3.8612
Epoch 10/10
313/313 ━━━━━━━━━━━━━━━━━━━━ 10s 32ms/step - accuracy: 0.1167 - loss: 3.7750 - val_accuracy: 0.1146 - val_loss: 3.8301
79/79 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.1106 - loss: 3.8189
```